

Pixel Battle — Visual Polish (Sub-project E) — Design Spec

Pixel Battle — Visual Polish (Sub-project E) — Design Spec

For agentic workers: This is a *design spec*. arlong pre-approved this sub-project (no Q&A); writing-plans generates the implementation plan next.

- **Date:** 2026-05-25
- **Status:** Pre-approved by user. Run autonomously to completion (no review gates, no mid-task questions).
- **Relationship:** Sub-project E, follows D (timeline scripts). Engine remains untouched; this is a pure renderer upgrade.

1. Motivation

After Sub-project D the simulation is smooth and scripted. The user's verdict: “做這麼久 是有進步 但還沒有到我想要的樣子 我覺得你可以把 **格鬥天皇** 當模板 然後是更順暢的感覺.” Stick figures over a plain background look amateurish next to a polished fighting game. KOF's signature look is its **HUD framing** (top bars, name plates, round counter) and **impact feedback** (hit sparks, screen flash, KO splash). Adding those will move the perceived quality up a tier without needing real sprite art.

2. Goal

Wrap the existing engine + scripted-fight renderer in a KOF-style **HUD + impact-feedback layer**. Same characters, same physics, same scripted choreography — but framed and punctuated so the viewer reads it as a fighting game, not an animation experiment.

3. Scope & Constraints

In scope — four polish layers:

1. **HUD overlay** — top-of-frame: two health bars with character name plates + (optional) round indicator. The bars are the iconic KOF read.
2. **Beefier hit feedback** — per-hit sparks scale with damage; full-screen color flash on crit / special / ultimate; floating “HIT!” / “CRIT!” text at the hit point.
3. **KO sequence** — when HP hits 0: brief simulation slow-motion (engine receives smaller `dt_ms` for ~1 s), camera zoom-in on the loser, “K.O.” splash text fades in.
4. **(Time permitting) Post-process color grade** — moviepy pass after render: slight saturation boost + soft vignette. Only if implementation is trivial; skip if it fights ffmpeg.

Hard constraints: – **No engine changes.** engine/, characters, skills, status effects, hitstop — all frozen. This is pure renderer work. – **No new heavy dependency.** Native `pygame.draw` + `numpy` for the HUD/FX work. `moviepy` already exists in the parent `Alvideo` project for the (optional) post-process pass. – **Visual safety preserved.**

`test_all_poses_keep_feet_planted_and_in_frame` must stay green — we are drawing OVER the world layer, not changing character geometry. – **No new characters / sprites / skills.**

Out of scope (deferred): – Real sprite art for characters (would take weeks of asset work). – Round system (best-of-3, round transition cards). Each scripted fight is still one round. – Voice cues (“FIGHT!”, “K.O.!”) — requires ElevenLabs work, separate sub-project. – Background parallax / animated background. – Combo counter on screen.

4. Current state (verified)

- `pixel_battle/rl/play.py::_render_fight` ticks the engine and composes the world surface each frame. Action source is now driver-agnostic (D-G3). The function receives a `FrameRecorder` and a per-tick `action_source(env, obs)` callable.
- `pixel_battle/engine/battle.py` emits `Event` objects on hits (`EventType.HIT`, `CRIT`, `ULTIMATE_START`, `KO`, etc). The renderer already consumes some of these (impact bursts, landing dust, flash puffs).
- `pixel_battle/rl/stick_renderer.py` has `spawn_impact_burst`, `spawn_landing_dust`, `spawn_flash_puff` — small particle helpers. This sub-project extends the same idiom.

- `BattleState.KO` is set when a character's HP reaches 0. `_render_fight` already holds the final frames for `end_hold_frames` ticks — that's where the KO splash + zoom lives.

5. Architecture — three new renderer modules

The engine and `_render_fight` orchestration stay the same shape; three new modules layer effects on top.

- **`pixel_battle/rl/hud.py`** — HUD class. Stateless per-frame draw: `HUD.draw(surf, battle, time_ms)`. Top of frame: two horizontal health bars with character name plates (KOF-style: bar drains right-to-left for left fighter, left-to-right for right fighter, both meeting in the middle). A vertical separator + a small clock or round indicator between them. Colors come from the character data (each character already has a brand color from earlier sub-projects). Health bar fills smoothly (lerped) so HP changes read as motion, not as a snap.
- **`pixel_battle/rl/impact_fx.py`** — owns a list of active impact effects + handles spawning them from engine events. Three primitives:
 - **Big spark burst** — `spawn_hit_spark(world, x, y, damage, color)` — a radial blast of 8–16 short bright lines scaling with damage (more lines + longer for bigger hits).
 - **Screen flash** — `flash_screen(world, color, alpha)` — full-frame SRCALPHA rectangle overlay; ticked down each frame. Spawns on crit / special / ultimate / KO.
 - **Floating text** — `spawn_floating_text(world, x, y, text, color)` — “HIT!”, “CRIT!”, “K.O.!” — rises 30px over ~400 ms while fading out.
- **`pixel_battle/rl/ko_sequence.py`** — small state machine that takes over the last seconds of a fight. Triggers when `battle.state` becomes KO. States:
 - **IMPACT** (0–200 ms): full-screen white flash + “K.O.!” text spawn + screen shake.
 - **SLOW_M0** (200–1200 ms): engine receives `dt_ms / 3` for one second of sim time (≈ 333 ms of sim $\rightarrow$ renders as 1 s on screen). Camera zooms toward the loser's position over the same duration.
 - **HOLD** (remaining `end_hold_frames`): camera stays zoomed, KO splash text holds at full opacity, world freezes.
- **`pixel_battle/rl/play.py::_render_fight`** — composition only:

- Construct a HUD, an `impact_fx` registry, and a `ko_sequence` controller before the main loop.
- Each tick: read new `battle.events`; route hit events to `impact_fx`; route the KO event to `ko_sequence`.
- The KO controller decides `dt_ms` (slow-mo factor) and the camera zoom level for this frame.
- After drawing the world layer (existing code), composite the HUD on top, then composite the floating-text + screen-flash impact effects on top of that.

6. KO sequence — slow-motion mechanics

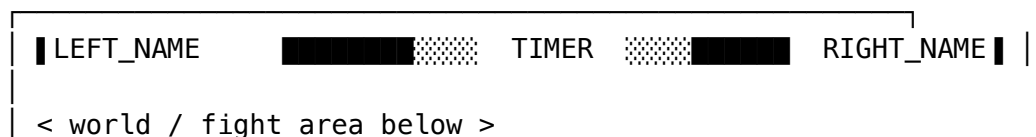
The engine's `tick_ms(dt)` is the clock. Calling it with a smaller `dt` advances simulation time more slowly. The renderer always outputs at 60 fps wall-clock — so passing `dt = ENGINE_TICK_MS // 3` during the slow-mo window means 1 second of wall-clock playback shows ~333 ms of in-game time. The hits the loser was already taking, the body falling, the dust kicking up — they all stretch out cinematically.

The slow-mo window starts at the KO event and lasts ~1 second of wall-clock, then the engine is essentially frozen (`dt = 0`) for the `HOLD` segment while the camera zooms and the splash text holds.

The camera zoom is implemented via the existing `CAM_ZOOM` mechanism — the controller writes a per-frame zoom value (lerping from 1.0 → 1.6 over the `IMPACT+SLOW_MO` window). The view rect re-centers on the loser's `pos_x`.

7. HUD — design details

Top 70 px of the 480 × 854 frame is the HUD strip. Layout:



- Health bars: 180 px wide, 16 px tall, drained from the inside outward. Drain animation lerps over 250 ms so flat damage reads as motion. Bar color is the character's brand color; backing is a dark muted version of the same.
- Name plates: 14-pt sans, dropshadowed for readability over any background.
- Center: a compact "TIMER" display showing match elapsed time MM:SS (small, dim) — KOF has a round timer; we don't have rounds, but the

timer reads as “fighting game” instantly.

8. Error handling

- Each new module’s draw call must no-op gracefully if its dependencies are missing (e.g. font init failure → skip text, don’t crash).
- KO sequence: if the engine never reaches `BattleState.KO` within the match (script exhausts, both alive), the controller stays in its initial `INACTIVE` state and the render simply ends at `max_seconds`. No slow-mo, no splash.
- Impact FX: a hit event with no actor field (defensive — shouldn’t happen) is silently dropped, no crash.

9. Testing

- **HUD:** drawn pixels match expected positions for both bars (left vs right alignment) and the timer; smooth-drain animation interpolates over 250 ms.
- **Impact FX:** `spawn_hit_spark` adds the correct number of particles per damage tier; `flash_screen` adds the expected `SRCALPHA` value; `spawn_floating_text` rises and fades according to its lifetime.
- **KO sequence:** the state machine transitions `IMPACT` → `SLOW_MO` → `HOLD` on the right schedule; in `SLOW_MO` the controller returns a `dt_scale` of $\sim \frac{1}{3}$; the camera zoom progresses `1.0` → `1.6`.
- **Integration:** all 5 timeline scripts still render to mp4; file sizes are in the same order of magnitude; no test goes red.
- **Visual safety:** `test_all_poses_keep_feet_planted_and_in_frame` stays green (we never touch character geometry).

10. Validation

Render all 5 timeline scripts. Confirm by playback: 1. Top HUD reads instantly as “fighting game” — bars drain visibly as HP drops. 2. Each hit shows a bigger spark + (on crits/specials/ultimates) a full-screen color flash. 3. The KO moment plays out in slow-motion with a “K.O.” splash and a zoom on the loser. 4. The whole thing still reads as our pixel-battle aesthetic — not an over-stylized parody.

11. Future work

- Real sprite-art characters (the actual KOF look).
- Round system + round-transition cards.
- Voice cues (“FIGHT!”, “K.O!”, round announcer).
- Background parallax.
- Combo counter HUD element.
- Post-process color grade as a separate optional pass.